Curso de Tecnologia em Sistemas de Computação
Disciplina: Programação Orientada a Objetos (EAD-05.032)
AD1 – 1º semestre de 2026.
Aluno: Douglas Neves

Decisões Arquiteturais - Sistema Smart Home

1. Visão Geral do Projeto

O Sistema de Monitoramento de Energia Residencial é uma aplicação orientada a objetos que gerencia cômodos e dispositivos eletrônicos em uma residência, permitindo monitorar e calcular o consumo de energia elétrica. A arquitetura foi projetada seguindo princípios de orientação a objetos **SOLID**, como visto em uma das aulas do Curso, com separação clara de responsabilidades entre os diferentes componentes.

2. Estrutura do Projeto

O projeto segue uma hierarquia de responsabilidades:

- **SmartHome** (Residência) → **Comodo** (Cômodos) → **Dispositivo** (Aparelhos eletrônicos)

Essa estrutura reflete a realidade física: uma residência contém vários cômodos, e cada cômodo contém vários dispositivos.

A Classe MenuSmartHome serve para ser a interface entre o usuário e o programa. Por fim, a Classe Main serve para testes, criação de instâncias, testar possíveis problemas, extrapolar o código no caso. Por fim, essa classe chama o MenuSmartHome para o usuário fazer testes manualmente.

3. Classe Dispositivo

3.1 Propósito e Responsabilidades

A classe Dispositivo representa um aparelho eletrônico individual. É a unidade mais básica do sistema e responsável por gerenciar seu próprio estado e consumo de energia.

3.2 Atributos

Atributo	Descrição
Nome (String)	Identificação do dispositivo
consumoHora (int)	Consumo de energia em kW por hora
TempoHoraUso (int)	Tempo acumulado de uso em horas
Ligado (boolean)	Estado do dispositivo (ligado/desligado)

3.3Métodos e Justificativas

ligar()

Permite ligar o dispositivo. Inclui validação para impedir ligar um dispositivo já ligado, mantendo consistência no estado.

desligar()

Permite desligar o dispositivo. Inclui validação para impedir desligar um dispositivo já desligado.

registrarTempoUso(float horas)

Registra tempo de uso do dispositivo. Contém duas validações importantes:

(1) Verifica se dispositivo está ligado,

(2) Valida se o tempo é maior que zero.

calcularConsumoHora()

Calcula o consumo total do dispositivo multiplicando consumo por hora pelo tempo de uso. Esse método está aqui porque apenas Dispositivo conhece seus próprios dados.

3.4 Por que NÃO está em outras classes?

- Não está em **Comodo**: Cada dispositivo é independente
- Não está em **SmartHome**: SmartHome não precisa conhecer detalhes individuais
- **Responsabilidade Única: Dispositivo só cuida de si mesmo**

4. Classe Comodo

4.1 Propósito e Responsabilidades

A classe Comodo representa um espaço físico em uma residência. É responsável por gerenciar a coleção de dispositivos dentro dele e coordenar operações em nível de cômodo.

4.2 Atributos

Atributo	Descrição
nome(String)	Nome do cômodo (ex: Sala, Quarto)
dispositivos[]	Array que armazena referências aos dispositivos
numeroTotalDispositivos(int)	Quantidade atual de dispositivos
capacidadeMax(int)	Limite máximo de dispositivos permitidos

4.3 Métodos e Justificativas

adicionarDispositivo(Dispositivo dispositivo)

Adiciona um dispositivo ao cômodo. Valida:

(1) Se dispositivo com mesmo nome já existe,

(2) Se cômodo não atingiu capacidade máxima.

OBS: Este método está em Comodo (não em SmartHome) porque é específico de cômodo: verifica capacidade local e duplicatas locais.

calcularConsumoTotal()

Calcula o consumo total de **todos os dispositivos** do cômodo. Está aqui porque Comodo é responsável por gerenciar seus dispositivos.

buscarDispositivo(String nomeDispositivo)

Localiza um dispositivo pelo nome dentro do cômodo. Responsabilidade de Comodo facilitar acesso aos seus componentes.

4.4 Por que NÃO está em outras classes?

- Não está em **Dispositivo**: Um dispositivo não gerencia coleções de cômodos.
- Não está em **SmartHome**: SmartHome não precisa de validações de cômodo
- **Responsabilidade Única: Comodo gerencia apenas seus dispositivos.**

5. Classe SmartHome

5.1 Propósito e Responsabilidades

A classe SmartHome representa a residência como um todo. É a classe de maior nível de abstração e responsável por gerenciar a coleção de cômodos e realizar operações globais em toda a residência.

5.2 Atributos

Atributo	Descrição
comodos[]	Array que armazena referências aos cômodos
numeroComodos(int)	Quantidade atual de cômodos cadastrados
capacidadeComodosMax(int)	Limite máximo de cômodos permitidos
taxaEnergia(int)	Constante: taxa de conversão kWh → Reais (1.15)

5.3 Métodos e Justificativas

cadastrarComodo(String nome, int capacidadeDispositivosMaxima)

Cadastra um novo cômodo à residência. Valida:

(1) Se cômodo com mesmo nome já existe,

(2) Se residência não atingiu capacidade máxima.

buscarComodo(String nome)

Localiza um cômodo pelo nome. Necessário para operações encontrarem o cômodo desejado.

calcularConsumoTotal()

Calcula o consumo **TOTAL da residência** somando consumo de todos os cômodos. Delega para cada Comodo calcular seu próprio total, depois agrega os resultados.

calcularCustoTotal()

Calcula o custo financeiro total multiplicando consumo total pela taxa de energia. Está aqui porque a taxa de energia é propriedade da residência.

exibirResumoDia()

Exibe relatório detalhado de toda a residência: consumo por dispositivo, por cômodo, e totais. Está em **SmartHome** porque agrega informações de toda a residência.

listarMaioresConsumidores()

Lista os dispositivos com maior consumo. É operação global:

(1) agrega todos os dispositivos da residência,

(2) ordena usando bubble sort,

(3) exibe ranking.

Não pode estar em Comodo porque compara dispositivos de cômodos diferentes.

5.4 Por que NÃO está em outras classes?

- Não está em **Comodo:** Operações de SmartHome são globais
- Não está em **Dispositivo:** Dispositivo não gerencia outros objetos
- R**esponsabilidade Única: SmartHome coordena e gerencia aspectos globais**

6. Classe MenuSmartHome

6.1 Propósito e Responsabilidades

MenuSmartHome é responsável **unicamente** pela interação com o usuário. Implementa a separação de UI (*User Interface*) da lógica de negócio.

6.2 Atributos

Atributo	Descrição
scanner	Objeto para ler entrada do usuário
smartHome	Referência à residência para executar operações

6.3 Separação de Responsabilidades

SEM separação: SmartHome teria código de Scanner, impressão de menus, etc. Seria impossível testar lógica sem interação com usuário, e difícil mudar para Interface Gráfica.

COM separação: SmartHome é classe 'pura' de lógica de negócio. MenuSmartHome coordena interface. Fácil de testar e modificar interface quando necessário.

6.4 Métodos e Justificativas

exibirMenu()

Objetivo simples de enviar todas as possibilidades para o usuário selecionar. Usa o Scanner para identificar a resposta do usuário e o Switch para direcionar para a opção selecionada.

lerInteiroComErro() e lerFloatComErro()

Método feito com a ajuda da IA Generativa quando ocorreu problemas de enviar números com vírgulas ou com o ASCII diferente por conta do teclado. Importou o java.util.Locale para usar o inputMismatchException para garantir que qualquer entrada numérica do usuário seja válida e tratada.

Os outros métodos são a interface entre o usuário e os métodos de SmartHome, Comodo e Dispositivos. Pequenas validações antes de entrarem nos métodos ou após retornarem com as respostas. Como por exemplo no buscarComodo que tem diferentes retornos ao usuário dependendo se não encontrar ou encontrar o cômodo.

7. Classe Main

7.1 Propósito

Main pode ter dois padrões:

(1) Ponto de entrada que cria SmartHome, cadastra dados de exemplo e exibe relatórios (demonstração), ou

(2) Ponto de entrada que cria SmartHome e MenuSmartHome, iniciando loop de menu interativo.

A responsabilidade de Main é CONECTAR os componentes, não implementar lógica de negócio.

Nesse caso, a Main faz os dois, depois de rodar a demonstração ela inicia o Menu deixando o usuário usufruir do sistema. Nesse momento que o problema com os números foi observado, inclusive.

8. Validações Aplicadas

8.1 Validações de Duplicatas

- SmartHome: Verifica se cômodo com mesmo nome já existe
- Comodo: Verifica se dispositivo com mesmo nome já existe

8.2 Validações de Capacidade (Limites)

- SmartHome: Número de cômodos < capacidadeComodosMax
- Comodo: Número de dispositivos < capacidadeMax

8.3 Validações de Estado (Lógica de Negócio)

- Dispositivo.ligar(): Não permite ligar dispositivo já ligado
- Dispositivo.desligar(): Não permite desligar dispositivo já desligado
- Dispositivo.registrarTempoUso(): Não permite registrar tempo se desligado

8.4 Validações de Entrada

- Dispositivo.registrarTempoUso(): Valida se tempo > 0

9. Regras de Negócio

Regras de negócio são restrições que garantem consistência e realismo:

- **Nomes únicos por escopo:** Não existem dois cômodos com mesmo nome. Não existem dois dispositivos com mesmo nome **NO MESMO cômodo**.
- **Capacidades limitadas:** Cada residência tem limite de cômodos. Cada cômodo tem limite de dispositivos. Reflete limitações físicas.
- **Tempo requer dispositivo ligado:** Não pode registrar tempo em dispositivo desligado. Garante consistência: tempo passa apenas se ligado.
- **Taxa de energia única:** Conversão de consumo (kWh) em custo usa taxa única (1.15 R$/kWh) para toda residência.
- **Estado binário:** Dispositivo está ligado ou desligado. Não há estados intermediários.

10. Princípios de Orientação a Objetos Aplicados

10.1 Responsabilidade Única

- Dispositivo: Muda se comportamento individual mudar
- Comodo: Muda se gerenciamento de cômodo mudar
- SmartHome: Muda se operações globais mudarem
- MenuSmartHome: Muda se interface com usuário mudar

10.2 Encapsulamento

Cada classe gerencia seus próprios dados. Dispositivo calcula seu consumo, Comodo busca seus dispositivos, SmartHome busca seus cômodos.

10.3 Coesão

Métodos e atributos dentro de uma classe estão fortemente relacionados. Todos os métodos de Dispositivo trabalham com estado de Dispositivo.

10.4 Composição

SmartHome é composto de Comodos. Comodos são compostos de Dispositivos. Hierarquia natural que reflete a realidade.

11. Evidências

```
douglas@douglas-desktop:~/Documentos/UFF/CEDERJ/Programacao Orientada a Objetos/AD1_DouglasNeves$  cd /home/douglas/Documentos/UFF/CEDERJ/Programacao\ Orientada\ a\ Objet
os/AD1_DouglasNeves ; /usr/bin/env /usr/lib/jvm/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /home/douglas/Documentos/UFF/CEDERJ/Programacao
\ Orientada\ a\ Objetos/AD1_DouglasNeves/bin Main
Sistema de Monitoramento de Energia Residencial

Cômodo 'Sala' cadastrado com sucesso!
Cômodo 'Quarto' cadastrado com sucesso!
Cômodo 'Cozinha' cadastrado com sucesso!
Cômodo 'Banheiro' cadastrado com sucesso!

 Tentativa de cadastrar cômodo duplicado! Não iremos cadastrar novamente.
Cômodo 'Sala' já existe na residência!

Dispositivo 'TV - Douglas - Polo Niterói' adicionado ao cômodo Sala.
Dispositivo 'Ventilador - Douglas - Polo Niterói' adicionado ao cômodo Sala.

 Tentativa de registrar duplicado!
Dispositivo 'TV - Douglas - Polo Niterói' já existe no cômodo Sala.

Dispositivo 'Ar Condicionado' adicionado ao cômodo Quarto.
Dispositivo 'Luminária' adicionado ao cômodo Quarto.

 Tentativa de registrar duplicado!
Dispositivo 'Ar Condicionado' já existe no cômodo Quarto.
Dispositivo 'Geladeira' adicionado ao cômodo Cozinha.
Dispositivo 'Micro-ondas' adicionado ao cômodo Cozinha.

 Tentativa de registrar duplicado!
Dispositivo 'Geladeira' já existe no cômodo Cozinha.
Dispositivo 'Chuveiro Elétrico' adicionado ao cômodo Banheiro.
Dispositivo 'Secador de Cabelo' adicionado ao cômodo Banheiro.

 Tentativa de registrar duplicado!
Dispositivo 'Chuveiro Elétrico' já existe no cômodo Banheiro.
```

```
Cômodo: Quarto
  Dispositivo: Ar Condicionado
  Estado: Ligado
  Horas de uso: 8.0
  Consumo diário: 6,40 kWh
  Dispositivo: Luminária
  Estado: Desligado
  Horas de uso: 0.0
  Consumo diário: 0,00 kWh
  Consumo do cômodo: 6,40 kWh

Cômodo: Cozinha
  Dispositivo: Geladeira
  Estado: Ligado
  Horas de uso: 12.0
  Consumo diário: 0,48 kWh
  Dispositivo: Micro-ondas
  Estado: Desligado
  Horas de uso: 0.0
  Consumo diário: 0,00 kWh
  Consumo do cômodo: 0,48 kWh

Cômodo: Banheiro
  Dispositivo: Chuveiro Elétrico
  Estado: Ligado
  Horas de uso: 0.5
  Consumo diário: 0,25 kWh
  Dispositivo: Secador de Cabelo
  Estado: Desligado
  Horas de uso: 0.0
  Consumo diário: 0,00 kWh
  Consumo do cômodo: 0,25 kWh

Consumo total da residência: 8,03 kWh
Custo total do dia: R$ 9,23
```

```
Dispositivo TV - Douglas - Polo Niterói ligado.
Dispositivo Ar Condicionado ligado.
Dispositivo Geladeira ligado.
Dispositivo Chuveiro Elétrico ligado.

Tempo de uso registrado: 4.0 horas.
Tempo de uso registrado: 8.0 horas.
Tempo de uso registrado: 12.0 horas.
Tempo de uso registrado: 0.5 horas.
Tempo de uso registrado: 2.0 horas.

 Tentativa de registrar tempo de uso em um dispositivo desligado!
Dispositivo Ventilador - Douglas - Polo Niterói está desligado. Não é possível registrar tempo de uso.

 Tentativa de registrar tempo inválido (negativo)!
Tempo de uso inválido: o valor deve ser maior que zero.

 Resumo do Dia:

Cômodo: Sala
  Dispositivo: TV - Douglas - Polo Niterói
  Estado: Ligado
  Horas de uso: 6.0
  Consumo diário: 0,90 kWh
  Dispositivo: Ventilador - Douglas - Polo Niterói
  Estado: Desligado
  Horas de uso: 0.0
  Consumo diário: 0,00 kWh
  Consumo do cômodo: 0,90 kWh
```

```
 Maiores Consumidores do Dia:
1) Ar Condicionado (Quarto) - 6,40 kWh
2) TV - Douglas - Polo Niterói (Sala) - 0,90 kWh
3) Geladeira (Cozinha) - 0,48 kWh
4) Chuveiro Elétrico (Banheiro) - 0,25 kWh

 Iniciando Menu Interativo:

Menu Smart Home:
1. Cadastrar cômodo
2. Adicionar dispositivo a um cômodo
3. Ligar dispositivo
4. Desligar dispositivo
5. Registrar tempo de uso de um dispositivo ligado
6. Exibir resumo do dia
7. Listar maiores consumidores
8. Listar capacidades de cômodos
9. Sair
```